Jijiga Express Super-App Upgrade Plan

Objective

Improve the existing Jijiga Express app into a polished regional super-app without removing the existing logistics, dispatch, tracking, inventory, accounting, EEU, parcel, ride, support, and security capabilities.

Current Baseline

The project is a React/Vite/TypeScript/Capacitor application backed by Firebase and Firestore. It already has Customer, Mamila/Vendor, Runner, Rider, and Admin roles; live maps; order status transitions; inventory reservation; delivery events; parcel and ride flows; EEU custody; accounting utilities; localization files; and regression/security tests.

The current customer experience is operationally capable but dense. The upgrade will add a clearer mobile-first discovery layer, strengthen food ordering and serviceability, and centralize customer-facing navigation without weakening server-side controls.

Implementation Decisions

1. Keep the existing one-vendor/single-batch persistence rules intact during the first safe increment. Introduce a cart abstraction in a backward-compatible way before enabling multi-item order persistence.
2. Add a customer-facing service grid and mobile navigation while retaining desktop navigation.
3. Add delivery-zone/serviceability as a client-facing validation layer first, with a clear path to Firestore/backend enforcement.
4. Preserve COD as the only currently trusted payment mode unless a signed payment backend is present.
5. Use existing English, Somali, and Amharic locale infrastructure for new customer-facing strings.
6. Keep Hotel behind a feature flag until its data model and booking rules are implemented.

Project Structure

- `src/App.tsx` : app shell, role routing, global order and inventory orchestration.

- `src/roles/CustomerView.tsx` : customer discovery, checkout, tracking, history, and profile.
- `src/roles/MamilaView.tsx` : vendor inventory and order handling.
- `src/roles/RunnerView.tsx` and `src/roles/RiderView.tsx` : dispatch and delivery operations.
- `src/roles/AdminView.tsx` : administration, reassignment, financials, and disputes.
- `src/components/customer/` : customer discovery and navigation components.
- `src/services/` : Firebase, authentication, maps, notifications, dispatch, and persistence.
- `src/types.ts` : domain types and status unions.
- `firestore.rules` : authorization, state-transition, and data-integrity controls.
- `scripts/` : regression and Firestore rule tests.

Upgrade Sequence

Phase 1: Customer foundation

- Add a compact mobile service grid.
- Add mobile bottom navigation for Home, Services, Orders, Help, and Profile.
- Keep desktop navigation available at larger breakpoints.
- Improve customer-friendly labels for internal order statuses.
- Centralize new customer-facing strings through locale support.

Phase 2: Food discovery and serviceability

- Keep the existing FoodDiscovery layer and connect it to live catalog/vendor data where possible.
- Add vendor metadata: rating, distance, ETA, delivery fee, minimum order, open/closed status.
- Add filter and sort controls for rating, delivery time, price, distance, and availability.
- Add a clear unsupported-location state and prevent checkout when the selected city/zone is not enabled.

Phase 3: Ordering core

- Introduce Cart and CartItem client models without breaking the current Order model.
- Support one vendor per cart for the first increment.
- Revalidate inventory, price, vendor status, quantity, address, and fees before order creation.

- Add idempotency protection against repeated checkout taps.
- Keep current Firestore price and reservation rules authoritative.

Phase 4: Authentication and retention

- Add phone OTP as a feature-flagged authentication path for +251 and +252.
- Preserve existing email/Google authentication for staff and existing users.
- Add loyalty points ledger and tier UI only after server-side ledger rules exist.
- Add a unified Help Center with categorized tickets linked to orders/bookings.

Phase 5: Validation and release

- Run typecheck, production build, unit regression, Firestore security tests, and focused customer-flow checks.
- Document deferred work, data migrations, security changes, and deployment steps.

Design Direction

Use an original Jijiga Express visual system: emerald/teal for trust and logistics, amber/orange for food, blue for parcel/maps, violet for admin, and slate neutrals for clarity. Prefer large readable text, thumb-friendly actions, clear empty/loading/offline/error states, and restrained transitions.

Required Safety Constraints

- Do not weaken Firestore rules.
- Do not trust client-side totals, roles, payment confirmations, or delivery completion.
- Do not remove existing specialized services.
- Do not introduce fake discounts or fake payment success.
- Do not expose secrets or private provider keys in client code.
- Do not claim completion for features that lack data-model, authorization, UI, and test coverage.